TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies
Department of Computer Systems

"[Jordan Thomas Brodie 243483MVEB]"

"[PROGRAMMING 1 BIG HOMEWORK]"

[Homework]

Supervisor: "[Risto Heinsar]"

Tallinn 2023

Author’s declaration of originality

I hereby certify that I am the sole author of this report. All the used materials, references to the literature and the work of others have been referred to.

Author: "Jordan Thomas Brodie"

[24.10.2024]

Table of contents

Author's declaration of originality 2
Table of contents 3
List of figures 4
List of tables 5
1 Task description 6
1.1 Task variant 7
2 Function $\boldsymbol{y = f(x)}$ analysis 8
2.1 Function $\boldsymbol{y = f(x)}$ plot 9
3 Solution description 10
3.1 Program flow 13
3.2 Algorithm 14
3.3 Special situation analysis 16
4 Summary 18
Appendix 1 – Screenshots 19

List of figures

Figure 1: My Function y = f(x) 8
Figure 2: Graph of function 9
Figure 3 : UML diagram of y= f(x) 15
Figure 4: Normal Operations of Program 19
Figure 5 : Example of an Unsolved Problem 19

List of tables

Table 1 : Outputs of the function..... 11
Table 2 : Special Situation Analysis..... 16

1 Task description

Tabulating a function y=f(x)

For homework I, you will have to create a software solution, that will solve a function y = f(x) within the limitations specified. The results must be displayed as a table with columns for X and Y values. Some hints about aligning it can be found from **printf tips example**.

The program will get the initial data from the keyboard (entered by the user) as real numbers!

Argument (x_i)	Function value $y_i=f(x_i)$
x_1	y_1
x_2	y_2
…	…
x_n	y_n

From your personal variant you will find the formulas for both x_i and y_i.

Function value should only be shown if the value exists. If the value cannot be calculated for the given x or is a complex number, You can choose to display “missing” or “complex”. If you wish, you are free to calculate the imaginary numbers and display them as well. Regardless of the variant, you will need to find up to 20 results.

The inputs for the program come from the keyboard and is entered by the user. All inputs besides the number of steps N must support real numbers. Number of steps N is an integer.

You will need to create and submit a program written in C, an algorithm for the program represented as an UML activity diagram and a report.

1.1 Task variant

In my variant, the user enters four values: A (starting point), B (stopping point), H (step size), and C (coefficient). The program uses these to calculate x values, starting from A and increasing by steps (H) and (C). The calculations stop when x becomes greater than B.

User inputs a starting value A, stopping value B, step H and step's coefficient C.
The conditions $\boldsymbol{B} > \boldsymbol{A}$ and $\boldsymbol{H}, \boldsymbol{C} > \mathbf{0}$ have to be true.
The function value y will be calculated in the following points while $\boldsymbol{x} \leq \boldsymbol{B}$:
A
A + H
A + H + CH
A + H + CH + C^2H
A + H + CH + C^2H + C^3H
...

2 Function $y = f(x)$ analysis

My function takes a value for x and calculates a number y based on the formula see Figure 1. The formula involves square roots, cubes, and division.

- If the denominator of the function equals to 0 then the function breaks because you can't divide by zero (this happens when x= 2 or x= -2)
- If the square root part involves a negative number, we get complex numbers, which the program detects and handles.

This function calculates y using square roots and division but has limits on what values x can have.

$$y = \frac{\left(3 + \sqrt{1 + \sqrt{x^3}}\right)}{(4 - x^2)}$$

Figure 1: My Function y = f(x)

These are the domain and range of my function.

Domain : $x \in [0,2) \cup (2,\infty)$

Range : $y \in (0,\infty)$

2.1 Function $y = f(x)$ plot

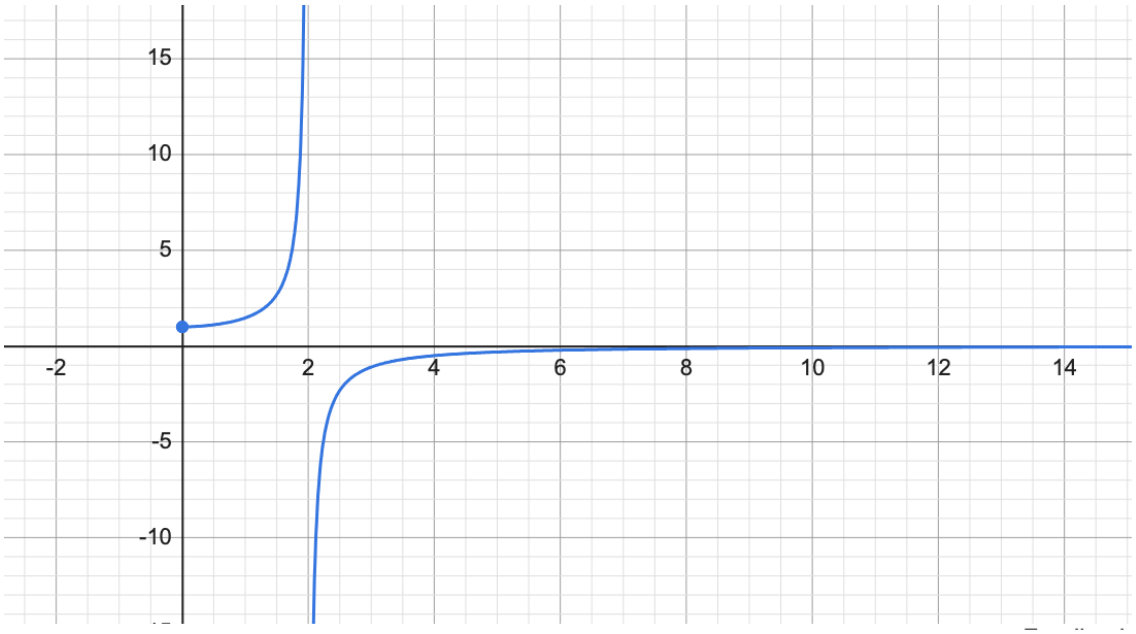


Figure 2: Graph of function

The graph of this function has certain restrictions. When x is near 2 or -2 (or when denominator will equal 0), the function has a gap because it can't be calculated (As you can see from the graph) There are places where the function either gives complex values or goes undefined as you can see from the starting point at x = 0.

3 Solution description

What is my program's purpose?

The purpose of my program is to calculate and display values for the function:

$$y = \frac{\left(3 + \sqrt{1 + \sqrt{x^3}}\right)}{(4 - x^2)}$$

What does my program do?

The program takes user input, performs calculations, and shows the results in a table. It also checks for errors and handles cases where the function can't be calculated or the result is complex.

Describe what data the user needs to enter and how does the user input behave. In which formats is the input expected in?

The program asks the user to enter: Four different integers.

A Starting value which is A, The first value of x to start calculations.

The Stopping value which is B, Where to stop calculating. It has to be greater than the value of A

The Step size which is H, how much to add to x for each step.

And lastly, the Coefficient C which is used to adjust the step size.

All values need to be entered as floating-point numbers (decimals), except the number of steps, which is an integer. If A, B, H, or C are negative, the program stops and tells the user the inputs are wrong.

Explain how is the data kept in your program – what data is stored, which data types are used and why?

In the program:

The input values A B H, and C are stored as floats.

X is calculated and stored temporarily as a float

Y is also stored as a float and printed as they are being calculated.

Using floats ensures that we can use decimal numbers in the program.

Explain the output of your program – which formats are used to print the results. Remember to cover all possible result types that your program can have!

The output is shown in a table, like this:

Table 1 : Outputs of the function

X	Y
1.0000	0.8750
1.5000	missing
3.0000	complex

It will print out as many values as possible but no more then 20 values all dependent on the conditions the user inputed A, B, H, and C.

The program inputs missing when the function does not work for that value of X.

It will also output complex, when the number is the square root of a negative number or a complex number.

If you created any user-defined functions (besides main()), add a paragraph that describes them .For each function ,describe their name, purpose, arguments and return value. NB! Creating user-defined functions is not among the topics for homework 1!]

I created a function called “calculate” to do the calculations.

Function Name: “calculate`

- Purpose: This function calculates the values of y for each x value and prints the results.
- Arguments:
    - Float A starting value for X
    - Float B Stopping Value
    - Float H Step Size
    - Float C coefficient that adjust the step
- Return Value, if It doesn’t return anything it just prints the table.
- Conclusion: The program calculates and prints the values for the function based on the user’s input. When the function can't be calculated it displays either "missing," or "complex." If not it prints the results.

3.1 Program flow

- Start the program
- Ask user to Enter Starting Value to start the program
- Ask the user to enter the Stopping Value
- Ask the user to enter H the steps
- Ask the user to enter the Coefficient
- While user is inputting these values make sure that A is greater then 0, B is greater then A, H and C are both positive numbers, if any input is wrong display an error message.
- Start calculating the values for x using the formula.
- Start at Starting Value and increase by steps based on H and C.
- Keep going as long as x is less then or equal to the stopping value.
- For each value of x calculate using our function.
- If the denominator is zero, print "missing."
- If the calculation involves square roots of negative numbers, print "complex."
- Otherwise, print the result of y with 4 decimal places.
- Show the results in a table with two columns one for x and one for y.
- End the program when calculations are done or if an error occurs.

3.2 Algorithm

Here’s an outline of the Algorithm, a breakdown of the UML diagram.

1. START (Initial Node)

2. Input value A (Action Node)

3. Input value B (Action Node)

4. Input value H (Action Node)

5. Input value C (Action Node)

6. Check if B > A and H, C > 0 (Decision Node)

- If true, proceed to the next step.

- If false, display "Invalid inputs!" and end the program

7. Initialize Variables (Action Node)

- Set value for x and start result counter i = 0

8. Loop: Calculate and Display Results (Loop Node)

- For each value of x, the following happens.

1. Check if 0 in denominator (Decision Node)

- If true, display "missing" for y.

-then go to next step 2.

- If false, check the next condition.

2. Check if x < 0 (Decision Node)

- If true, display "complex" for y

- then go to next step 3

- If false, calculate the and display the result.

3. Update X and if it hasn't computed over B times or 20 times

-If true, go back to beginning of loop

Increment result counter

-If False, go to 9 and exit loop

9. End the program

SEE Figure 3. Below for Graphical version of this Diagram.

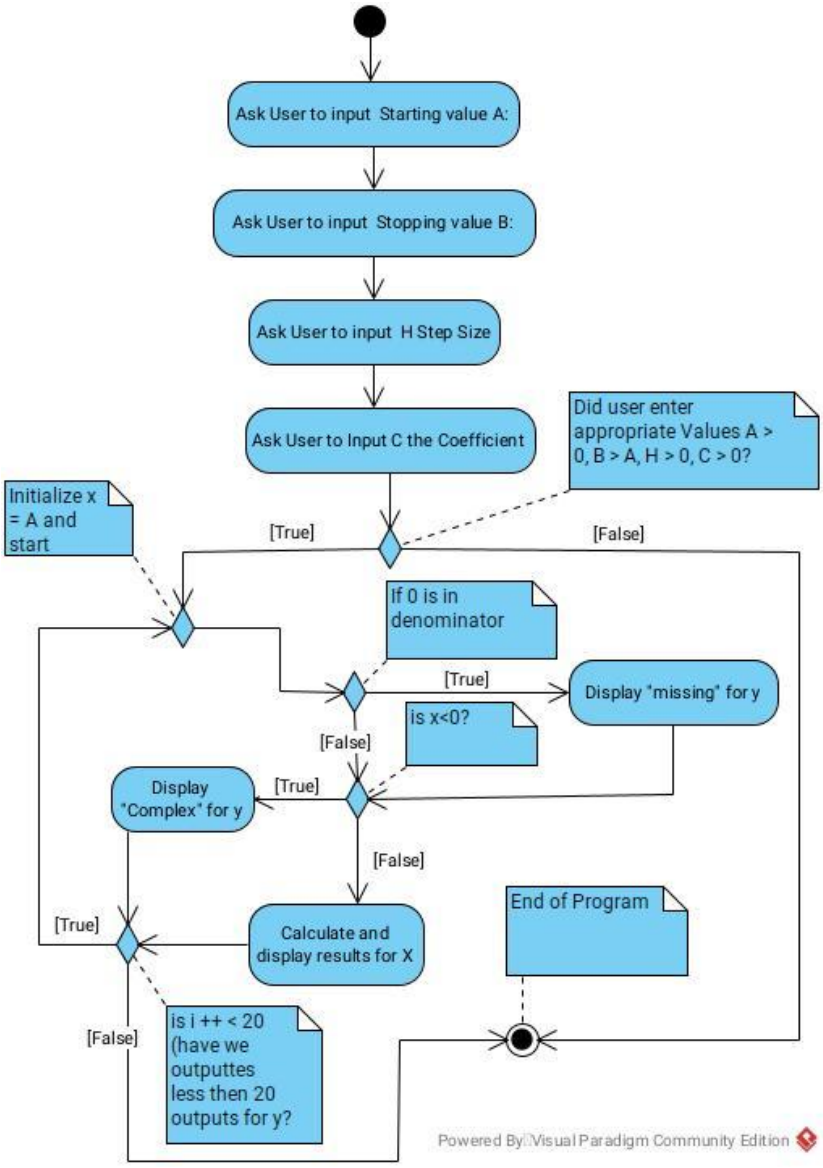


Figure 3 : UML diagram of y= f(x)

3.3 Special situation analysis

In this section, I will go over potential errors or special situations that may occur while the program runs, how they affect the program, and whether the issue has been addressed. For issues that haven't been solved, I will propose potential solutions. Below is a table summarizing the possible situations:

Table 2 : Special Situation Analysis

Situation	Problem Description	Mitigated/Unsolved	Proposed Solution
Input for B <=A	If the stopping value is less than or equal to the starting value, the range for x-values will be invalid, and the loop won't run.	Mitigated	Program checks this and if the input is incorrect the user is informed with a message.
Negative or zero step size	A step size less then or equal to zero will cause program to malfunction.	Mitigated	The program checks this, if input is invalid, user gets a message.
Negative Coefficient	If the coefficient C is negative or zero the formula will keep getting the same x values.	Mitigated	The program checks if C is greater then 0 if it input is invalid program stops and user is informed.
If there is a zero in the denominator when calculating function	This will lead to an undefinded value of y	Mitigated	The program detects this condition, and if so it prints missing, avoiding errors.
Square root of a negative number.	This would result in a complex number which can't be calculated	Mitigated	The program checks for negative values inside square roots and if so it prints complex.
More than 20 results.	Since we are dealing with functions with infinite outputs for y. The program may not stop.	Mitigated	The program limits the number of result to a maximum of 20 so it doesn't have an infinite amount of outputs.
Non Numeric Input	The user enters a non numeric value for A, B, H, or C it may crash the program.	Unsolved	I could write a loop that allows people multiple tries when entering values for A, B, H, or C.

Discussion of Table.

Mitigated Problems: These situations are handled in the current implementation in the program

Unsolved Issues : Are not resolved in this program but I do see how they can be resolved.

4 Summary

How I approached the task

I started by reading the task instructions and breaking them down step by step. First, I thought about how the inputs and outputs should work. I thought about things we've done before so far in class. Then, I focused on writing the code to calculate the function values. To be honest I had to talk to some friends and get help with google a lot to find some of the logic. I couldn't figure out. I just took it step by step.

Issues I had

I got stuck in the decimal places, and the alignment of the table. I also get confused with loops a lot. Since I am so reliant on google and friends. I feel like I don't have everything memorized so I am nervous about rearticulating this on a test.

Time it took:

It took me a few hours to make the code. The report and the UML code honestly took the most time probably 3 hours.

Additional thoughts

This task was not as hard as some of the tasks we are now doing in class so it felt a bit refreshing to go back, I can see how we are working on more complicated things now, and how progressive it is. I also saw how my knowledge of wear we are at now, can help me solve problems. I would re-write the code with the solutions solved such as the looper if someone inputs a non-numerical value for A, B, H, or C.

Appendix 1 – Screenshots

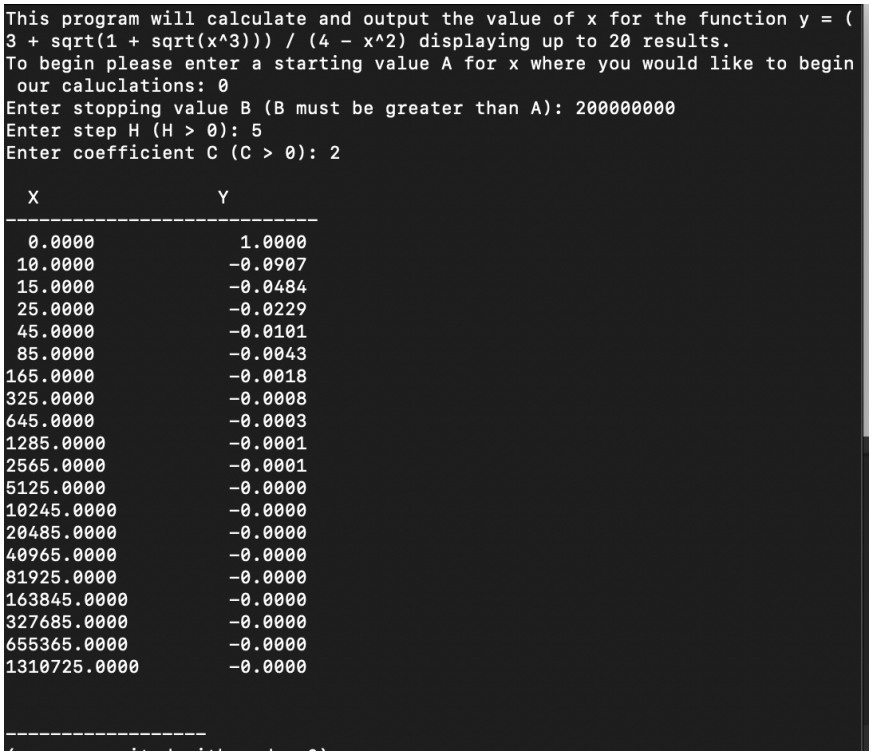


Figure 4: Normal Operations of Program

Figure 4. Shows a normal operation of Code.

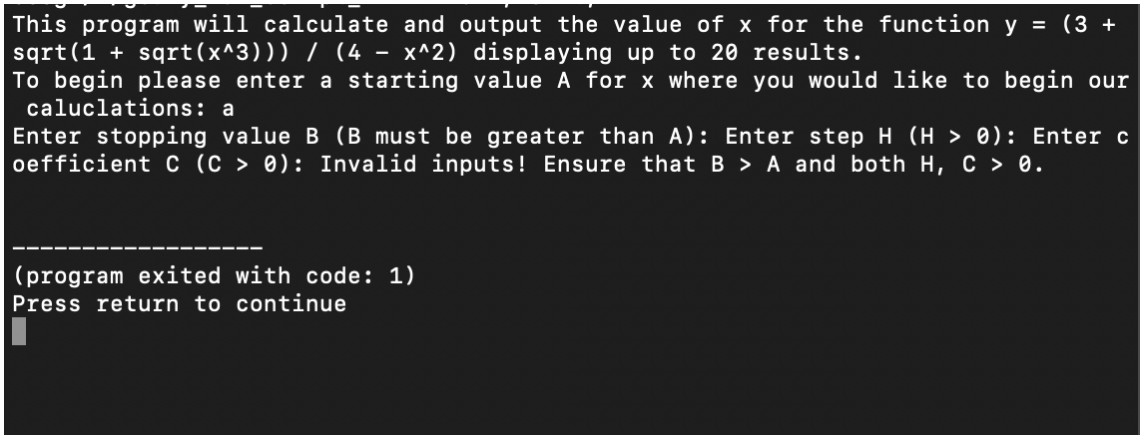


Figure 5 : Example of an Unsolved Problem

This Sample Figure 5 shows, how the program closes, when entering a non-numerical value for A. An alternative loop can be added to allow user other opportunities to enter correct value.